Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 20

Using third-party libraries

Università Bocconi
MILANO

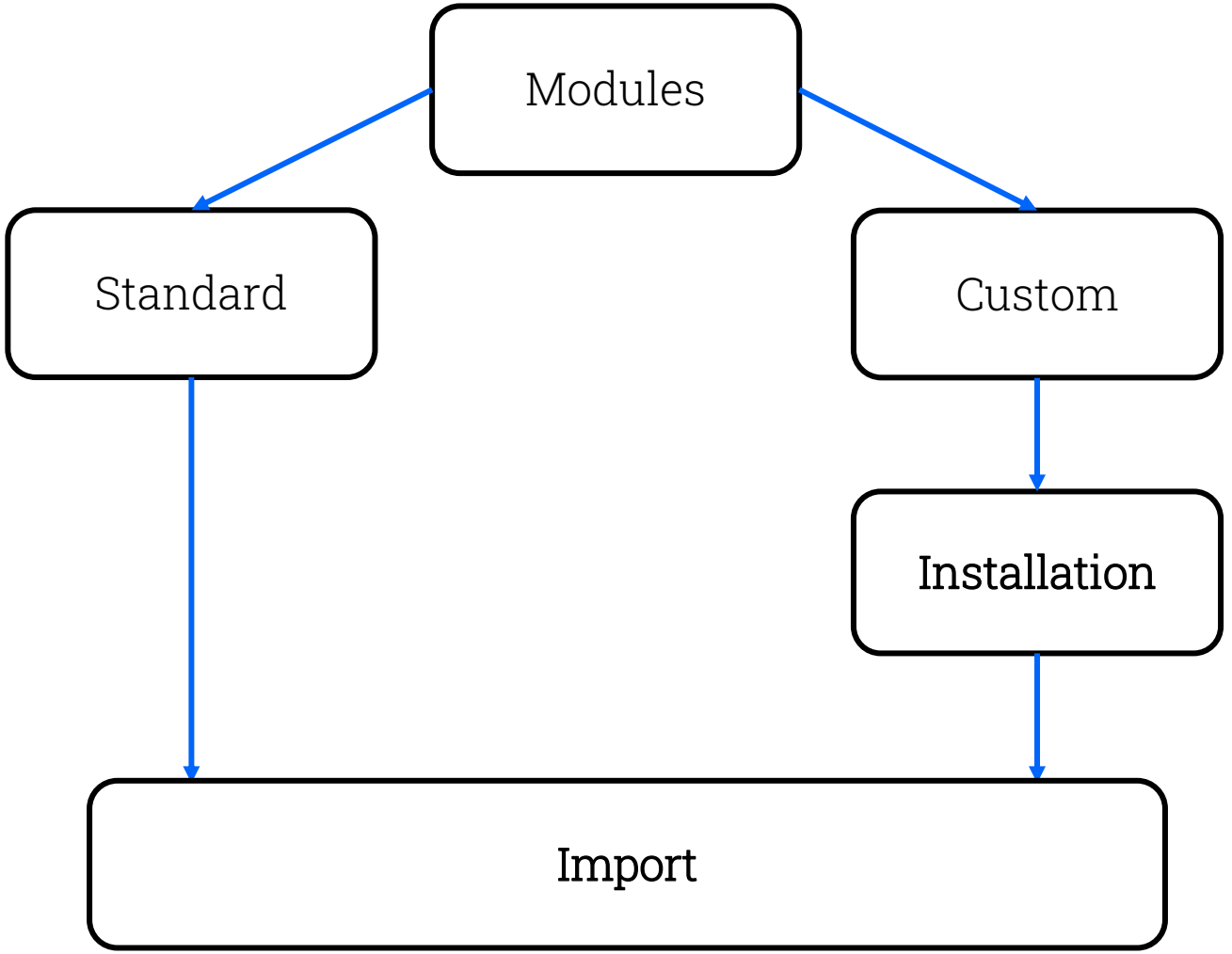
Objectives of the lesson

- Introduce additional modules of the standard libraries
- Install and use third party modules

The modules

- All programming languages can be enriched by extensions in order to execute specialized additional functionalities
- In Python, additional functionalities come as **modules**: these are files acting as containers, grouping useful functionalities by topic
- Some modules are part of the basic Python installation, and we already introduced some examples, such as *math* and *random*
- Modules can be organized in **libraries** or «packages»
- Modules installed by default with Python belong to Python's **standard library**
- There are also many custom modules useful for many different purposes

Standard and custom modules



Importing modules

Assuming a module is installed, in order to use it we need to **import** it into the program's running session.

If you use a module's functionality before you import it, an error is raised:

```
>>> math.sqrt(9)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    math.sqrt(9)
NameError: name 'math' is not defined. Did you forget to import 'math'?
```

The import command allows to import one or more modules, also using custom names, or to import only single functions:

```
>>> import math
>>> math.sqrt(25+9)
5.830951894845301
```

```
>>> import math, random
>>> math.pi * random.randint(10,20)**2
452.3893421169302
```

```
>>> import random as rnd
>>> rnd.randint(10,20)
14
```

```
>>> from random import choice
>>> choice([1, 3, 5, 7])
3
```

Note: each module is only available in the work session in which it is imported!

Examining modules

The dir command shows the list of the loaded modules (the ones always available in Python, and those imported in the current work session):

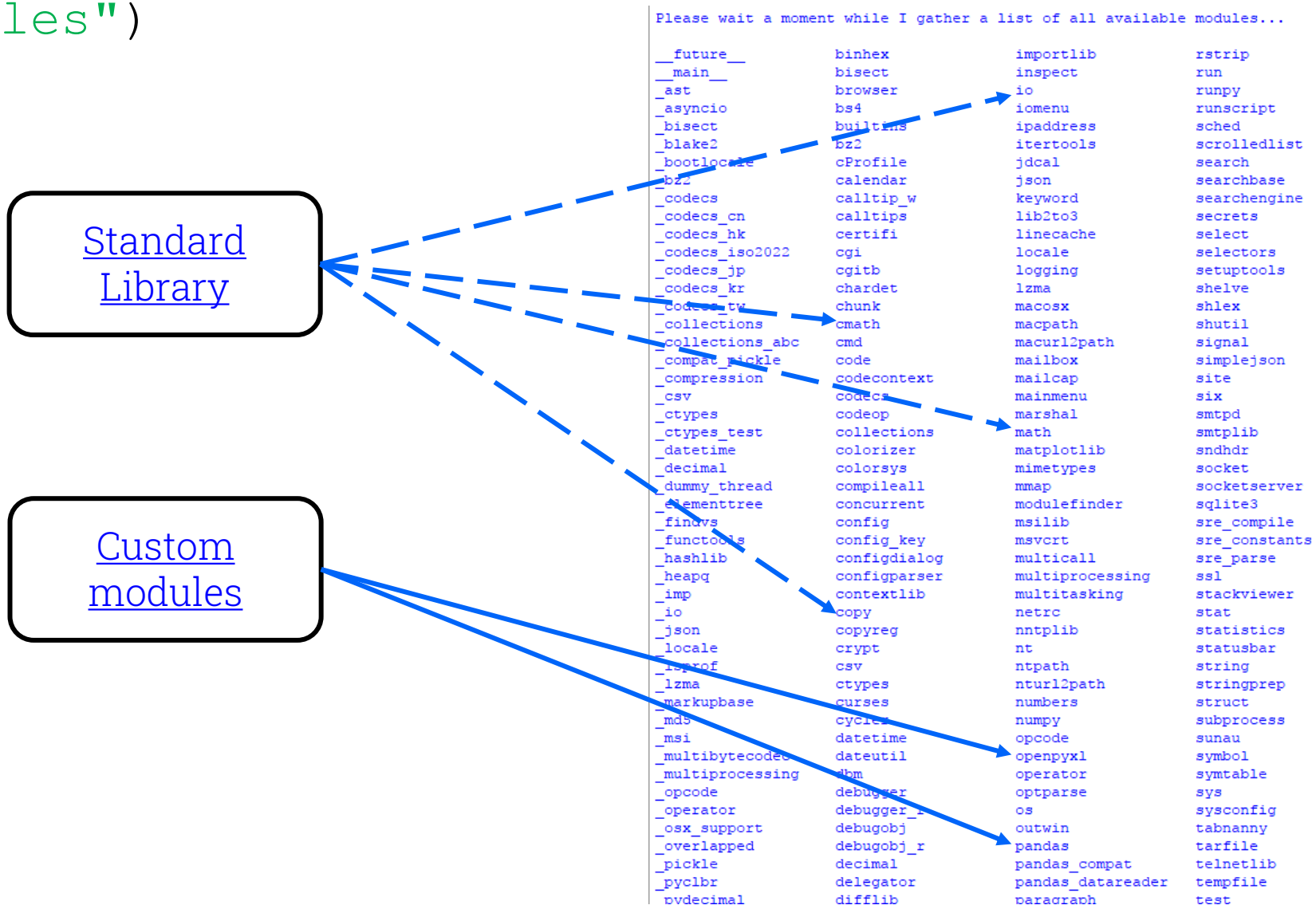
```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__']
>>> import math
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
'__package__', '__spec__', 'math']
```

The same command also allows listing the functions and classes offered from a specified module:

```
>>> dir(math)
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos',
'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', '
cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'fac
torial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf'
, 'isclose', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log
10', 'log1p', 'log2', 'modf', 'nan', 'pi', 'pow', 'radians', 'sin', 'sinh
', 'sqrt', 'tan', 'tanh', 'tau', 'trunc']
```

Listing all the available modules

```
help("modules")
```



Università Bocconi MILANO

Modules help

It is possible to use **help** function to get information about modules:

```
>>> import math
>>> help(math)
Help on built-in module math:

NAME
    math

DESCRIPTION
    This module is always available.  It provides access to the
    mathematical functions defined by the C standard.

FUNCTIONS
    acos(...)
```

It is also possible to apply **help** to single functions contained in a module:

```
>>> help(math.floor)
Help on built-in function floor in module math:

floor(...)
    floor(x)

    Return the floor of x as an Integral.
    This is the largest integer <= x.
```

Not working if the module has not been imported!

Modules from the standard library

During the course we have used the following functions from modules of the standard library:

- **math** (to provides access to the mathematical functions)

`math.sqrt(x)` ⇨ returns the square root of x as float

`math.pi` ⇨ is the value of Pi (i.e. 3.141592653589793)

- **random** (to generate random data)

`random.random()` ⇨ returns a random float in the interval [0, 1), were 0 is included and 1 is excluded

`random.randint(a, b)` ⇨ returns a random integer in range [a, b], including both end points

`random.randrange(start,stop=None,step=1)` ⇨ returns a random item from range(stop) or range(start, stop[, step])

`random.choice(seq)` ⇨ returns a randomly chosen element from a non-empty sequence

https://docs.python.org/3/library/math.html

https://docs.python.org/3/library/random.html

Modules from the standard library

Another interesting module to explore is **webbrowser**, a Web-browser controller that provides a high-level interface for launching and remotely controlling web browsers:

- **webbrowser** (to open web pages in the default browser)

```
open(url, new=0)
```

Display the web page available at **url** using the default browser:

- if **new** is **0**, the url is opened in the same browser window
- if **new** is **1**, a new browser window is opened
- if **new** is **2**, a new browser page ("tab") is opened

https://docs.python.org/3/library/webbrowser.html

webbrowser module: example

In the following script, the **wiki** function uses the **webbrowser** module to open, using the default browser, the Wikipedia page containing the keyword passed to the function as an argument:

```
import webbrowser

def wiki(KW):
    '''KW = the keyword of the search you want to perform
    on Wikipedia (to be inserted in quotation marks).
    If the term is composed, use "_" instead of space'''

    webbrowser.open('https://en.wikipedia.org/wiki/'+KW)
```

webbrowser module: exercise

Create a program that asks the user to enter a series of keyword for a query to launch with **Google**. The program must ask the user to enter the keywords one after the other. When the user have no other keyword to enter, the program must launch Google's webpage in the default browser.

Note: to create the program check carefully how the URL of a Google search is made.

```
import webbrowser

MyQuery = ''
MyKW = input('Type the keyword you want to search: ')

while MyKW != '':
    MyQuery = MyQuery + MyKW + '+'
    MyKW = input('Do you want to enter another keyword?\n\
    (Type the new keyword or hit Enter key to launch the search): ')

webbrowser.open('https://www.google.com/search?q='+MyQuery)
```

Custom modules

Specialized modules, usually intended for niche audiences, that must be installed by the user.

Where to find them?

PyPI - **the Python Package Index** is the official site collecting and categorizing most of the available modules

There are many other sources offering information and download of modules. For instance, UsefulModules contains a list of the most popular Python modules classified by topic, mainly aimed at a Python beginners.

Installing custom modules

To use a custom module it is necessary to **download** and **install** it, executing a specific command from the command-line prompt of the operating system:

Windows OS	Mac OS
In the Command Prompt window (cmd):	In the Terminal window:
`pip install <modulename>`	`pip3 install <modulename>`

openpyxl module

Non-standard Python module for reading and writing Excel files.

openpyxl allows to create, read, write, and update Excel files (.xlsx / .xlsm / .xltx / .xltm). It is especially useful for automating Excel-related tasks like data analysis, report generation, or working on spreadsheets with lines of code.

Key Features:

- open, create, and save Excel files in .xlsx format
- modify cell values, styles, formulas, and more
- add charts, images, and formulas
- access and update multiple sheets in a workbook

https://openpyxl.readthedocs.io/en/stable/

openpyxl module

Here are some useful commands and methods:

Command or method	Description
MyNewXLfile = openpyxl.Workbook ()	Creates the instance *MyNewXLfile* of the class of objects Excel workbook (Note: a new empty workbook has always one single worksheet)
MyNewXLfile.save ('filename.xlsx')	Saves the Excel workbook object as *filename.xlsx* on the active path
MyXLfile = openpyxl.load_workbook ('filename.xlsx')	Opens the existing workbook *filename.xlsx* and returns it as an Excel workbook object (creating the instance *MyXLfile*)
MyXLfile.create_sheet ('WSname')	Creates the new worksheet *WSname* at the end of *MyXLfile* workbook
MySheet = MyXLfile['WSname']	Create the instance *MySheet* of the worksheet object *WSname* of *MyXLfile*
MyXLfile['MySheet']['A1'] = NewValue	Writes *NewValue* in cell *A1* of *MySheet* worksheet of *MyXLfile* workbook
MyValue = MySheet.cell(x,y).value	Returns the value of the cell at coordinates x,y of *MySheet* object (creating *MyValue*)
R = MySheet.max_row C = MySheet.max_column	Return the maximum row index and the maximum column index containing data in *MySheet*

***Note**: all Blue Labels are instances of objects of different classes*

openpyxl module: example

In the following script the **openpyxl** module allows to print on the screen the list of companies present in an Excel file:

```
import openpyxl

MyWorkbook = openpyxl.load_workbook('companies.xlsx')
MyWorksheet = MyWorkbook['Feb']
idx = 7
companies_list = []

while MyWorksheet.cell(row=idx, column=3).value != None:
    companies_list.append(MyWorksheet.cell(row=idx, column=3).value)
    idx = idx + 1

print('\nCompanies:\n')
for company in companies_list:
    print(company)
```

Output:

```
Companies:

Improved Hardware Company
Supreme Shoe Company
New Doorbell Company
Crisp Electronics Corporation
New Doorbell Company
Different Ink Inc.
Fine Tripod Company
Fine Tripod Company
Bright Adhesive Company
Innovative Shovel Partners
Improved Hardware Company
Bright Saddle Corporation
Crisp Banister Inc.
Exclusive Edger Traders
Stunning Raft Supply
Guaranteed Luggage Corporation
Bright Adhesive Company
Amazing Glass Inc.
```

pandas module

Non-standard Python module for data analysis and statistical analysis.

pandas allows to create two-dimensional *dataframe* objects (which can be represented as a dataset) importing data from:

- external sources (i.e. Excel, csv, txt, etc.)
- Python objects of different data types (e.g. dictionaries of lists, lists of lists, etc.)

It is also possible to create the dataset from scratch

https://pandas.pydata.org/docs/user_guide/index.html

pandas module

Here are some useful commands and methods:

Command or method	Description
MyDF = pandas.DataFrame(data=D)	Create the instance *MyDF* of a class of two-dimensional, size-mutable dataframe objects. The constructor method can create empty instances or generate datasets based on arrays, constants, dictionaries, lists, etc. (e.g. the instance *D* in the example) or import data from external sources
MyX = pandas.read_excel('filename', 'sheet')	Creates the instance *MyX* of a pandas dataframe object, reading data from the *sheet* worksheet of the *filename* Excel file. Many optional arguments are available to customize the import process
.read_html() .read_sql() .read_csv() .read_xml() *(…etc.)*	Similar to ".read_excel", they are functions to create dataframe objects reading data from various sources
MyDF.to_dict(orient='x')	Converts the dataframe to a dictionary. The type of the key-value pairs in the output can be customized with the *orient* argument: 'dict' (default) ⇨ dict like output: {column : {index : value}} 'list' ⇨ dict like output: {column : [values]} 'records' ⇨ list like output: [{column : value}, … , {column : value}]

Exercise 1

We need to import the “Lesson20.xlsx” file in Python and create a dictionary to store the contents of the **Invoices** worksheet, using the basic Python functions and tools. In particular, we need to:

- import the contents of the Invoices worksheet of the “Lesson20.xlsx” file in a new file named **Exercise 1.py**, saving first the Excel worksheet as a Text (Tab-delimited) *.txt file
- create a dictionary in which the keys are the headers of the table, while the values are lists containing all the elements of the corresponding columns

Exercise 2

We need to import the “Lesson20.xlsx” file in Python and create a dictionary to store the contents of the **Invoices** worksheet, using the **openpyxl** module functions and tools. In particular, we need to:

- import the contents of the “Lesson20.xlsx” file (the table in the Invoices worksheet) in a new file named **Exercise 2.py** using the functions and tools of the **openpyxl** module
- create a dictionary in which the keys are the headers of the table, while the values are lists containing all the elements of the corresponding columns

Exercise 3

We need to import the “Lesson20.xlsx” file in Python and create a dictionary to store the contents of the **Invoices** worksheet, using the **pandas** module functions and tools. In particular, we need to:

- import the contents of the “Lesson20.xlsx” file (the table in the Invoices worksheet) in a new file named **Exercise 3.py** using the functions and tools of the **pandas** module
- create a dictionary in which the keys are the headers of the table, while the values are lists containing all the elements of the corresponding columns

Tools to learn with today's lecture

- Understand the difference between standard modules and custom modules
- Learn how to use functionalities of some additional standard modules
- Learn how to find and install external libraries
- Install and use the modules openpyxl and pandas

Book references

Understanding Python:

Chapters 12.1, 12.2, 12.3, 12.4, 12.10, 13.1, 13.2, 13.3, 13.7.1, 13.7.2